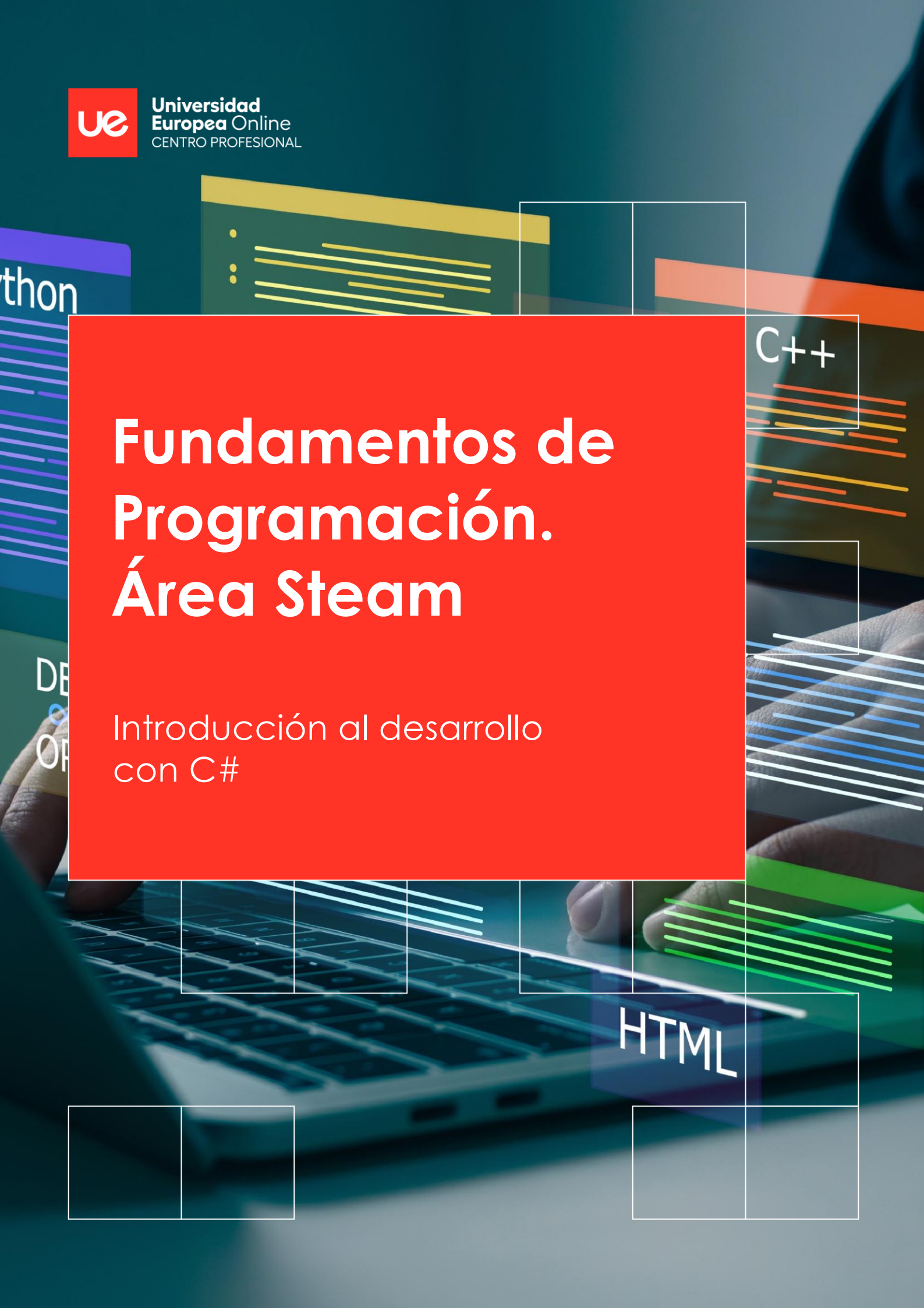
Universidad Europea Online
CENTRO PROFESIONAL



Fundamentos de Programación. Área Steam

Introducción al desarrollo con C#

Índice

Presentación 3
Objetivos de aprendizaje 3
1. Introducción a la programación 3
2. ¿Qué es un algoritmo? 4
3. Desarrollo *software* 5
   3.1. Ciclo de vida del *software* 5
   3.2. Metodologías 6
4. Lenguajes de programación 8
5. Primeros pasos con C# 9
   5.1. Instalaciones 10
   5.2. Primer proyecto 10
Resumen 13
Referencias bibliográficas 13

© Centro de Formación Profesional Europeo de Madrid

Presentación

La programación, un pilar esencial de la ciencia de la computación, nos permite entender cómo funcionan los ordenadores y su capacidad para ejecutar instrucciones. Este proceso, aunque complejo, está lleno de desafíos que hacen que los resultados sean muy satisfactorios. Para ello, es necesario tener una sólida comprensión de la programación, entender qué son los algoritmos y el proceso que conlleva la creación de un programa.

Los lenguajes de programación nos permiten "hablar" con la máquina en un lenguaje similar al natural, con estructuras determinadas. Existen múltiples lenguajes de programación y plataformas hoy en día para crear *software*. La elección correcta depende de factores como el ámbito de uso (móvil, escritorio, web), la plataforma donde se ejecutará (Windows, Mac, Android, iOS, web) o la ubicación del código de programación (cliente o servidor).

La plataforma .NET de Microsoft, a través de su lenguaje de programación C#, es una de las soluciones más populares en el mundo. Esto se debe a la posibilidad de realizar un desarrollo multiplataforma y a la adaptabilidad que tiene el código desarrollado para diferentes escenarios. Este tema se centrará en explorar estos conceptos y herramientas en profundidad.

Objetivos de aprendizaje

Los objetivos que se pretenden alcanzar en este recurso son los siguientes:

- Conocer el concepto de programación.
- Entender el concepto de algoritmo.
- Conocer las diferentes metodologías de desarrollo *software*.
- Instalar y configurar los IDE necesarios para desarrollar en C#.

1. Introducción a la programación

La programación se ha convertido hoy en día en uno de los aspectos básicos en cualquier campo de la sociedad industrial. Esta afirmación se basa en el hecho de que todas las empresas necesitan programas y *software* específicos para poder llevar a cabo sus tareas cotidianas, por lo que desarrollarlos y adaptarlos a las necesidades que cada una tiene, es básico. Entender esto es ver la necesidad de comprender conceptos de cómo poder crear dichos programas, su manera de interactuar con el usuario y la forma en la que se gestiona la información internamente.

Hoy en día no entendemos una empresa sin ordenadores y estos, a su vez, deben ser utilizados por los usuarios y, no solo de la forma correcta, sino con las especificaciones técnicas para que el trabajo que deben realizar se haga de la mejor forma posible. Esto solo se consigue con el *software* adecuado para ello. Aquí es donde entra la programación.

© Centro de Formación Profesional Europeo de Madrid

Programar no es más que indicar las tareas o pasos que una máquina tiene que hacer para poder solventar un problema y llevar a cabo una ejecución concreta. Suena muy sencillo, pero no lo es tanto. Hay que tener en cuenta que cuando nosotros nos proponemos solucionar un problema, el proceso que conlleva hacerlo nos sale casi instintivo si este es relativamente sencillo o, si es más complejo, en muchas ocasiones hay parte del proceso que no tenemos que pensar. Esto se debe a que los seres humanos tenemos un pensamiento lógico y racional, que nos es innato. Sin embargo, cuando este problema tiene que ser solucionado por una máquina, es el desarrollador el que le tiene que indicar esos pasos, teniendo que hacerlo muy en detalle, ya que la naturaleza de la máquina es la ejecución de las líneas de código de una en una, no pudiendo dar nada por sentado. Es aquí donde el desarrollador debe tener ese pensamiento lógico del que hablábamos antes y ser capaz de expresarlo y codificarlo de forma correcta para que este pueda ser ejecutado por la máquina.

2. ¿Qué es un algoritmo?

Desarrollar un programar es comunicar al ordenador las instrucciones que tiene que realizar para poder llevar a cabo ejecuciones que solventen un problema. Para ello, es necesario crear "recetas" para poder indicar que es lo que es necesario hacer. Esto es lo que se conoce como algoritmo.

Un algoritmo es un conjunto ordenado y finito de operaciones simples que permiten solucionar un problema o tomar una decisión. Cuando un desarrollador quiere enfrentarse a un problema, lo primero que tiene que hacer no es escribir líneas de código sin más, sino enfrentarse al problema, proponiendo soluciones que puedan llevar a cabo su código a buen puerto.

Para poder hacer un algoritmo de forma correcta, debemos seguir los siguientes pasos:

1. Definición del problema. Este punto es básico para poder hacer un algoritmo no solo que funcione bien, sino que sea lo más eficiente posible. Entender bien lo que se pide y plantearlo de forma correcta es el primer punto para realizar la solución.
2. Diseño. Una vez se ha comprendido el problema, pasamos a ver las necesidades que tenemos para poder plantear la solución de este. En este punto tenemos las siguientes tareas:
    a. *Input / output*: detectar cuáles son los datos de entrada que necesitará el programa, como los datos de salida que ofrecerá.
    b. Descomposición: en informática hay una premisa muy importante que es: divide y vencerás. Lo mejor es dividir el problema en pequeños problemas que sean de fácil implementación, así el conjunto de todos ellos resolverá la tarea general.
    c. Ordenación: estructura todos los pasos para que la solución diseñada se ejecute de forma correcta y en el orden necesario, teniendo en cuenta los dos puntos anteriores.

Para poder dejar definida esta fase, se utiliza lo que más adelante se verá como diagramas UML, que no son más que una serie de documentos que ayudan a plasmar

© Centro de Formación Profesional Europeo de Madrid

todas las soluciones diseñadas para que sea más fácil su compresión y posterior implementación.

1. Implementación. Con el diseño realizado, pasamos a poner en marcha el algoritmo codificándolo, primero en pseudocódigo y luego en el lenguaje de programación correspondiente. Hay que recalcar que una buena implementación siempre se lleva a cabo si previamente se ha realizado un buen diseño y análisis del problema.

Importante

El pseudocódigo es un lenguaje intermedio entre el lenguaje natural y el lenguaje de programación de muy fácil compresión y no ejecutable, que permite comprender la lógica de la solución de forma sencilla.

2. Implantación y pruebas. Una vez se ha terminado de codificar es momento de probar el código en un entorno lo más real posible. En esta fase se depura el código, viendo determinados fallos en ejecución y solventándolos para el que la solución funcione de forma completa.

Ejercicio

Ahora te toca a ti plantear un par de algoritmos:

1. El primero de ellos será un ejemplo de la vida cotidiana. Sé todo lo detallista que puedas para poder crear una solución que se pueda implementar.
2. El segundo de los algoritmos será un ejemplo de un programa informático simple, como una suma de números, el cálculo de una potencia. No te preocupes por no saber aún códigos de programación, lo que se busca con este ejemplo es que intentes sacar la lógica del problema que planteas.

3. Desarrollo *software*

A pesar de lo que mucha gente fuera del mundo del desarrollo se piensa, hacer un *software* de calidad no es simplemente elegir un lenguaje de programación y empezar a tirar líneas de código hasta obtener el producto deseado. El proceso de desarrollo *software* engloba una serie de etapas que no pueden ser pasadas por alto si se quiere realizar un producto de calidad y con todas las especificaciones indicadas por el cliente. Por ello, es básico aplicar de forma correcta cada una de las fases, además de seleccionar de forma correcta cómo se va a realizar este proceso, o lo que es lo mismo, la metodología que seguirá el equipo de trabajo a la hora de llevar el proyecto.

3.1. Ciclo de vida del *software*

Cuando un cliente (externo o interno) requiere un *software* y se lo encarga a un equipo de desarrollo, este debe marcar las pautas para poder realizar cada uno de los pasos que conlleven la realización del *software*. En estos pasos es importante involucrar al

© Centro de Formación Profesional Europeo de Madrid

cliente, ya que es él quien debe indicar las funcionalidades del *software*, siempre teniendo en cuenta que el equipo de desarrollo tiene el conocimiento para poder hacer cada uno de los requisitos que se indica. Estas fases son las siguientes:

1. Planificación. Una vez el cliente expone su problema, el equipo analiza cuál es la solución viable para lo que se propone. Además de analizar el problema, se tienen en cuenta aspectos como costes, beneficios, recursos que se deben emplear, etc.
2. Diseño. Con todos los requisitos recogidos en la fase anterior, los ingenieros *software* diseñan y crean posibles soluciones para cada uno de requisitos.
3. Implementación. Se trata de la fase de codificación, donde el equipo de desarrollo empieza a escribir líneas de código que hagan realidad cada uno de los diseños que los ingenieros han realizado en el punto anterior.
4. Pruebas. Una vez las implementaciones son terminadas, es momento de probarlas y comprobar su funcionamiento.
5. Despliegue. En esta fase de producción, el *software* se hace accesible al cliente, pudiendo así utilizarlo con total normalidad.
6. Mantenimiento. Una vez el *software* se despliega, hay que mantener su funcionamiento de forma correcta, comprobando cada una de las funcionalidades.

Estas fases del ciclo de vida del *software* son el *standard*, pero dependerá de la metodología que se aplique.

3.2. Metodologías

Las metodologías de desarrollo de *software* son enfoques sistemáticos y estructurados que proporcionan pautas y procesos para llevar a cabo proyectos de desarrollo de *software* de manera eficiente y efectiva. Estas metodologías establecen la forma en que se planifica, ejecuta y controla todo el ciclo de vida del desarrollo de *software*, desde la concepción de la idea hasta la entrega del producto final.

El concepto de metodología en el desarrollo de *software* se refiere a un conjunto de principios, prácticas, procedimientos y reglas organizadas para guiar el proceso de creación, diseño, implementación y mantenimiento de *software*. Estas metodologías proporcionan un marco de trabajo que ayuda a los equipos de desarrollo a planificar, ejecutar y gestionar proyectos de manera eficiente, asegurando la calidad del producto final.

En el contexto del desarrollo de *software*, una metodología establece cómo se deben realizar las distintas etapas del ciclo de vida del *software*, desde la conceptualización de la idea hasta la entrega del producto. Esto incluye la definición de roles y responsabilidades, la asignación de tareas, la gestión de riesgos, las prácticas de desarrollo, las pruebas y la documentación.

Las metodologías pueden variar en enfoque y complejidad, y se adaptan a las necesidades específicas de cada proyecto. Algunas son más flexibles y ágiles, como SCRUM o Kanban, mientras que otras son más estructuradas y secuenciales, como el

© Centro de Formación Profesional Europeo de Madrid

modelo en cascada. La elección de una metodología particular depende de factores como el tamaño del equipo, la naturaleza del proyecto, los requisitos del cliente y la cultura organizacional.

Alguna de las principales metodologías son:

3.2.1. DEVOPS

DevOps es una filosofía y práctica que busca integrar el desarrollo de *software* (Dev) con las operaciones (Ops) para lograr una entrega continua y eficiente de *software*. Esta metodología se centra en la colaboración estrecha entre equipos, la automatización de procesos y la mejora continua. Hoy en día, el concepto de DevOps está muy de moda ya que otorga al proceso de desarrollo de una integración de la parte de desarrollo con la parte de sistemas.

Cuando se aplica una metodología DevOps sobre todo nos encontramos con cuatro conceptos claves:

- Colaboración. DevOps fomenta la comunicación y colaboración constante entre desarrolladores, operadores y otros profesionales involucrados en el ciclo de vida del *software*.
- Automatización. La automatización de procesos, desde la construcción hasta la implementación y las pruebas, acelera la entrega y reduce errores manuales.
- Entrega continua. DevOps busca lograr una entrega continua de *software*, permitiendo implementaciones frecuentes y rápidas.
- Monitoreo y retroalimentación. La monitorización constante del rendimiento y la retroalimentación rápida son esenciales para identificar problemas y realizar mejoras de manera proactiva.

3.2.2. SCRUM

Metodología centrada en la colaboración, flexibilidad y la entrega incremental de productos de alta calidad. A diferencia del resto metodologías, SCRUM marca cada hito en el proyecto como una fase completa, la cual incluye la gran mayoría de fases en del proceso de desarrollo, repitiendo estos hitos para cada una de las entregas de las que conste el proyecto. Esto hace que el desarrollo no sea estanco, es decir que para poder empezar la fase de pruebas sea necesario terminar la fase de desarrollo.

Los principios fundamentales de esta metodología son:

- Iterativo e incremental. SCRUM divide el proyecto en iteraciones llamadas *sprints*, generalmente de 2 a 4 semanas. Al final de cada *sprint*, se entrega un producto funcional e incrementado. Cada una de estas entregas marca una parte funcional del producto, donde se realizan funcionalidades incrementales
- Roles definidos. SCRUM define claramente los roles de los participantes para que cada uno realice sus tareas de forma correcta. Los principales son:
  - *Product Owner*: representa al cliente.

© Centro de Formación Profesional Europeo de Madrid

- *Scrum Master*: representa el "director" del proyecto.
- Equipo de desarrollo: profesionales encargados de realizar el trabajo.

3.2.3. En cascada

Esta metodología basa su ejecución de una serie de etapas claramente definidas y en un orden concreto, sin posibilidad de modificar la "hoja de ruta". Se trata de una metodología muy poco flexible, ya que no es posible pasar a la siguiente etapa sin haber completado la anterior

Se trata de una metodología que ya cuenta con varios años y, debido a su poca flexibilidad, no es muy adecuada para proyectos grandes ya que estos necesitan desarrollos iterativos y rápidos, por lo que tan solo se recomienda para proyectos pequeños que tienen un alcance muy definido y resultados que, a priori, no van a sufrir ninguna variación.

Desarrollar *software* es un proceso complejo que conlleva a mucha gente que debe ejecutar una serie de procesos claramente definidos para que el producto pueda llegar a entregarse a tiempo.

4. Lenguajes de programación

Como ya se ha comentado antes, los lenguajes de programación representan la manera en la que el desarrollador le transmite las ordenes al ordenador para que este las ejecute y se puedan realizar las tareas necesarias que permitan solventar un problema.

En el mundo existen infinidad de lenguajes de programación, ya que tú mismo puedes crear tu propio lenguaje, sin embargo, es un número limitado los que el mercado demanda año a año. Lenguajes como C#, Java, Python, JavaScript, TypeScript, C, HTML, Kotlin, o PHP son aquellos que siempre aparecen en los primeros puestos como los más demandador. No existe un lenguaje de programación perfecto, ni siquiera uno mejor que otro. Lo que realmente importa a la hora de decidir que lenguaje de programación utilizar en un proyecto, es el cometido de éste ya que depende del ámbito de aplicación unos lenguajes funcionan mejor que otros.

Importante

Es imposible controlar la sintaxis completa y funcionalidad de un lenguaje de programación, para eso están las documentaciones oficiales. Lo que sí es muy importante es dominar las bases del lenguaje, ya que de esta forma será mucho más sencillo comprender no solo el funcionamiento del lenguaje de programación en cuestión, sino también de muchos otros cuya estructura será muy similar.

Nosotros, en esta asignatura nos centraremos en C#, pero antes de empezar, es necesario que entendamos cuales son los diferentes tipos de clasificaciones de lenguajes que existen, así como su funcionamiento general.

© Centro de Formación Profesional Europeo de Madrid

En cuanto a las clasificaciones de los lenguajes de programación, existen diferentes formas de agruparlos. Para poder simplificar dichas clasificaciones vamos a basarnos en dos:

- Según su finalidad:
  - Lenguajes de bajo nivel: aquellos que están formados por un conjunto de instrucciones básicas (también llamados lenguajes ensamblador) o que están escritos en binario y se ejecutan directamente en la CPU (código máquina).
  - Lenguajes de alto nivel: aquellos que son de fácil lectura y creación por su similitud al lenguaje natural, lo cuales necesitan de un compilador o interprete para poder ser ejecutados.
- Según su forma de ejecutarse:
  - Lenguajes compilados: aquellos que son traducidos directamente a código máquina. Para ello, estos lenguajes necesitan de un programa extra llamado compilador que realiza el proceso de traducción. Algunos ejemplos de estos lenguajes son: C, C++, Rust, C#
  - Lenguajes interpretados: aquellos cuyo código fuente se ejecutar de forma directa por el procesador. Este tipo de lenguajes tiene un elemento llamado intérprete que se encarga de leer y ejecutar las líneas una a una. Algunos ejemplos de estos lenguajes son: Python, JavaScript, PHP



Existen algunos lenguajes que incorporar características de ambos tipos, como por ejemplo Java

Ejercicio



Realiza una investigación de cuáles son los lenguajes de programación más demandados hoy en día en el sector. Para ello puedes utilizar buscadores de empleo como Linkedin. Una vez los tengas indica los siguientes aspectos:

1. Clasifícalos según los grupos que hemos visto antes.
2. Cuál es su cometido principal.
3. En la vida real, en qué ámbitos se aplican.
4. Ejemplo de algún programa – web – *software* que lo utilice.

5. Primeros pasos con C#

En el mundo de la programación, contamos con infinidad de posibilidades a la hora de elegir un lenguaje de programación. En realidad, no es necesario conocer todos ellos (por no decir que es prácticamente imposible), sino lo que es interesante es ser experto en unos cuantos (sobre todo los que el mercado más demanda) y el resto se pueden ir aprendiendo con el paso de los años y las necesidades, ya que esto no será muy complicado al ser todos los lenguajes similares. En nuestro caso utilizaremos C# como

© Centro de Formación Profesional Europeo de Madrid

lenguaje de programación, pero antes de empezar con las instalaciones, es necesario que entendamos una serie de conceptos que iremos viendo a lo largo del curso:

- .NET: plataforma de desarrollo que permite la generación de *software* para diferentes sistemas operativos. No se trata del lenguaje de programación, sino el marco donde este se ejecuta. A grandes rasgos, estaríamos hablando del compilador del lenguaje
- C#: lenguaje de programación que puede utilizar la .NET para poder crear aplicaciones. Además de C#, .NET admite F# o Visual Basic como lenguajes de programación.
- IDE: se trata del entorno de desarrollo que será necesario para poder realizar los desarrollos mediante .NET y el lenguaje de programación seleccionado. Dependiendo del sistema operativo utilizaremos uno u otro. Para Windows se podrá utilizar Visual Studio y Visual Studio Code y para MacOs tan solo Visual Studio Code



Visual Studio dejó de tener soporte para MacOS en agosto de 2024.

Los *links* de descarga de cada uno de los IDES son los siguientes:

Visual Studio Code <https://code.visualstudio.com/>.

Extensiones Visual Studio Code <https://marketplace.visualstudio.com/items?itemName=ms-dotnettools.csdevkit&WT.mc_id=dotnet-35129-website>.

Visual Studio <https://visualstudio.microsoft.com/es/>.

5.1. Instalaciones

Dependiendo de cuál sea el sistema operativo de la máquina, utilizaremos un programa u otro. En ambos casos, el proceso de instalación es muy sencillo y tan solo requiere de seguir el asistente de instalación. En el caso de Visual Studio Code, tendremos que instalar de forma adicional la extensión C# Dev Kit para que, tanto la plataforma .NET como el lenguaje de programación, funcionen perfectamente.

5.2. Primer proyecto

Para poder crear el primer proyecto, dependerá del IDE que utilicemos.

- En el caso de hacerlo con Visual Studio Code, se puede hacer desde la paleta de comandos (menú ir -> paleta de comandos) e introducir .NET New Proyect. Si es el primer proyecto que se crea, se descargarán todas las dependencias de .NET necesarias. Tras esto, aparecerá un asistente donde se pedirán los siguientes datos
  - Tipo de aplicación: .NET tiene la capacidad de crear aplicaciones de diferentes tipos. En nuestro caso para empezar seleccionaremos Aplicación de consola
  - Ubicación del proyecto:

© Centro de Formación Profesional Europeo de Madrid

- Nombre de la aplicación.

- En el caso de hacerlo con Visual Studio, el proceso es muy similar, pero desde un asistente ubicado en la opción Create New Project o menú File -> New Project.

Con el proyecto creado, podremos observar que se han generado una serie de ficheros carpetas con la siguiente estructura:

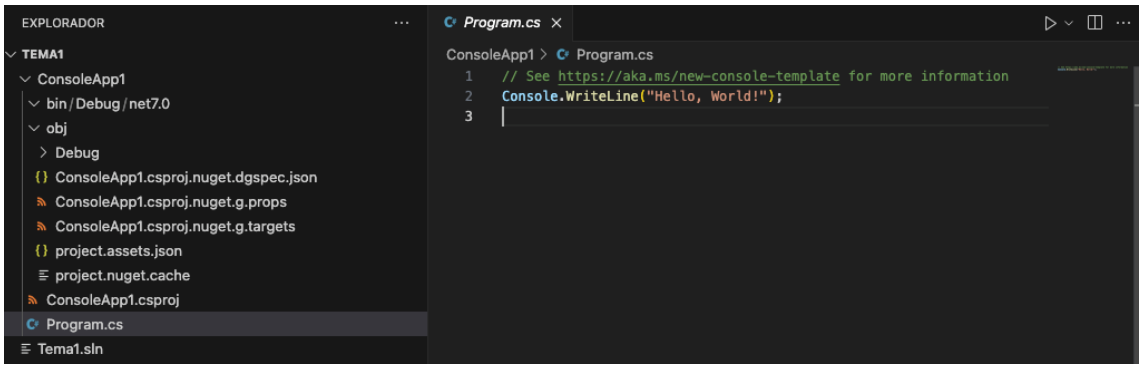


Figura 1. Estructura proyecto. Elaborada por Borja Martín Herrera.

Dado a que se generan gran cantidad de archivos, vamos a quedarnos con lo importante para empezar. En la raíz de la carpeta encontraremos un archivo con extensión .sln el cual representa la información de todos los proyectos que están ubicados en la ruta configurada. El otro de los ficheros importantes es el que tiene la extensión cs, el cual representa nuestro código, y es ahí donde crearemos todas las líneas de código que sean necesarias para realizar nuestro programa. Por defecto trae dos líneas, las cuales representan un comentario y una impresión por consola. Si ejecutamos la aplicación pulsando el símbolo del *play* ubicado en la parte superior derecha, veremos cómo aparecerá la consola de depuración del IDE con el que estemos trabajando y en ella se mostrará el mensaje Hello World.

Aunque más adelante en el curso se verán estos conceptos, vamos a definirlos brevemente para poder entender y crear algunas sentencias:

- Líneas de código: son todas aquellas instrucciones que son ejecutadas por la máquina. Estas pueden ser de diferentes tipos como definición de variables, llamadas a métodos, estructuras de control, etc.

- Métodos: todas aquellas funcionalidades que queremos que se ejecuten. Podemos decir que son acciones que se lanzan y pueden ser ya creadas (como el método WriteLine) o creadas por nosotros mismos. Los métodos son reconocibles porque siempre acaban con paréntesis, donde puede o no haber cosas dentro.

- Parámetros: son aquellos datos que una funcionalidad necesita para poder ejecutarse correctamente. Dependiendo del tipo de dato que sea necesario se pondrá con/sin comilla doble. En el caso del ejemplo, el parámetro es *Hello World* y, al ser una palabra, va entre comilla doble.

© Centro de Formación Profesional Europeo de Madrid

Importante

En C#, las líneas de código (salvo alguna excepción como la definición de estructuras de control), terminan con un ; . Esto le indica al compilador que esa línea no tiene ningún elemento que ejecutar más, por lo que puede pasar a la siguiente línea.

Existen algunas líneas de código que no se ejecutan por parte del compilador, simplemente son comentarios que ayudan al desarrollador a documentar el código y hacerlo más comprensible. Estas líneas comienzan con los caracteres // para una sola línea o /* lo que se quiera poner */ para poder poner varias líneas

Como se puede ver en el código del ejemplo, el método WriteLine tiene como parámetro una palabra, lo que permite escritura o impresión de esta en la consola de depuración. En el caso de querer poner otra frase, sería tan sencillo como llamar nuevamente al método, indicando lo que se quiera mostrar:

```
// See https://aka.ms/new-console-template for more information

Console.WriteLine("Hello, World!");

// Esto es ejemplo de comentario

Console.WriteLine("Bienvenido a la asignatura introducción a la programación");
```

En el caso de querer que el usuario introduzca datos por consola, se utiliza un método llamado Console.ReadLine().

```
Console.WriteLine("Por favor introduce alguna frase");

String lectura = Console.ReadLine();

Console.WriteLine(lectura);
```

En este caso, además de primero mostrar un mensaje, la consola se quedará esperando a que el usuario introduzca datos por teclado. Una vez pulsada la tecla *enter*, los datos introducidos serán guardados dentro de una variable llamada lectura y al mismo tiempo mostrados por consola (con el método WriteLine pasando como parámetro la variable que tiene los datos guardados).

Ejercicio

Con lo visto anteriormente, intenta ahora realizar una lectura de datos y su posterior mostrado. Para ello realiza lo siguiente:

- Después de mostrar un mensaje de bienvenida, pide por consola los siguientes datos:
  - Nombre
  - Apellido
  - Correo

© Centro de Formación Profesional Europeo de Madrid

- Dirección

- Una vez pedidos cada uno de estos datos, muestra por consola el siguiente mensaje: "Enhorabuena Juan López, te enviaremos un correo a juan@gmail.com con el comprobante", donde cada uno de los datos son los previamente introducidos.

Como *tip*, ten en cuenta que, para poder unir palabras dentro de una salida por consola, es necesario utilizar el carácter + como, por ejemplo: "Esto es una "+variable+" que ha sido concatenada".

Resumen

Durante este tema, hemos visto una primera aproximación a la programación, con un centrado en el desarrollo utilizando el lenguaje C#. Para poder comprender el concepto de programación, hay que tener en cuenta importancia de los algoritmos en la resolución de problemas, que no son más que la guía que debe seguir el programa que se codifica. Estos algoritmos deben seguir pensamiento lógico en la programación para que puedan ser ejecutados de forma correcta, solventando todas las tareas que inicialmente se proponen.

Una vez comprendido todo esto, hay que tener en mente que la programación no solo es escribir líneas de código, sino que también abarca diferentes etapas del ciclo de vida del *software* donde cada una de ellas cobra especial importancia para que todo el proceso cobre sentido y sea, no solo más sencillo desarrollar el producto final, sino también con la calidad correcta. Para ello, existen diferentes metodologías como SCRUM, DevOps, Cascade, etc., que permiten seguir este proceso de forma correcta.

Por último, se ha visto en el tema como poder descargar e instalar los IDE que nos permiten desarrollar nuestros programas. Estos dependerán del sistema operativo que utilicemos, siendo Visual Studio Code o Visual Studio los que se utilizarán durante el curso.

Referencias bibliográficas

C# <https://dotnet.microsoft.com/es-es/languages/csharp>.

Instalación Visual Studio. <https://learn.microsoft.com/es-es/visualstudio/install/install-visual-studio?view=vs-2022>.

.NET Visual Studio Code <https://learn.microsoft.com/en-us/dotnet/core/tutorials/with-visual-studio-code?pivots=dotnet-8-0>.

© Centro de Formación Profesional Europeo de Madrid

Universidad Europea Online
CENTRO PROFESIONAL

Todos los derechos de propiedad intelectual de esta obra pertenecen en exclusiva al © Centro de Formación Profesional Europeo de Madrid. Queda terminantemente prohibida la reproducción, puesta a disposición del público y en general cualquier otra forma de explotación de toda o parte de la misma.

La utilización no autorizada de esta obra, así como los perjuicios ocasionados en los derechos de propiedad intelectual e industrial del © Centro de Formación Profesional Europeo de Madrid, darán lugar al ejercicio de las acciones que legalmente le correspondan y, en su caso, a las responsabilidades que de dicho ejercicio se deriven.

Ve más allá